Principal Component Analysis (PCA)

In this practice, we carry out a Principal Component Analysis (PCA) to set of data taken from https://online.stat.psu.edu/stat505/lesson/11/11.3. This consists of 329 observation vectors in $\mathbb{R}^9$ that correspond to the rating of 329 communities. The entries of each vector correspond to the rating of the following criteria: 1. Climate and Terrain, 2. Housing, 3. Healthcare & Environment, 4. Crime, 5. Transportation, 6. Education, 7. The Arts, 8. Recreation, 9. Economics.

1. Compute and store in MATLAB the standardized matrix $\hat{X}$.

```
% Import text file into MATLAB using "Import Data" at the Menu
pcadata = readtable('places.csv');
% Convert the first 9 columns to a matrix (ignoring the last
   column)
X = table2array(pcadata(:, 1:9))';
% Transpose to get observations in columns

% 1. Construct matrix X_hat
X_mean = mean(X, 2); % Mean along each row (each feature)
X_centered = X - X_mean; % Center on mean
X_std = std(X, 0, 2);  % Standard normalization by n-1 and
   along rows
X_hat = X_centered ./ X_std; % Standardization
```

2. Compute the SVD of $\hat{X}$ using the `svd` command.

```
[U, S, V] = svd(X_hat); % SVD of the standardized matrix
```

3. Obtain the ratio between the accumulated variance captures by the first p principal components and the total variance, for all possible values of p.

we know that the fraction of total variance "captured" by the first p principal components is

$$\sum_{j=1}^{p} \sigma_j^2 \Big/ \sum_{j=1}^{9} \sigma_j^2,$$

where $\{\sigma_i\}_{i=1}^{9}$ are the singular values of $\hat{X}$. For this task, we have used the useful MATLAB command `cumsum`, which given a vector $a \in \mathbb{R}^m$, returns another vector $b \in \mathbb{R}^m$ where the elements b_p of the new vector are given by

$$b_p = \sum_{i=1}^{p} a_i.$$

```
singular_values = diag(S);
total_variance = sum(singular_values.^2);
accumulated_variance = cumsum(singular_values.^2) /
   total_variance;
```

```
4
5  for p = 1:length(accumulated_variance)
6      fprintf('Captured variance by first %d principal components
           : %.4f\n', p, accumulated_variance(p));
7  end
```

This code produced the following output:

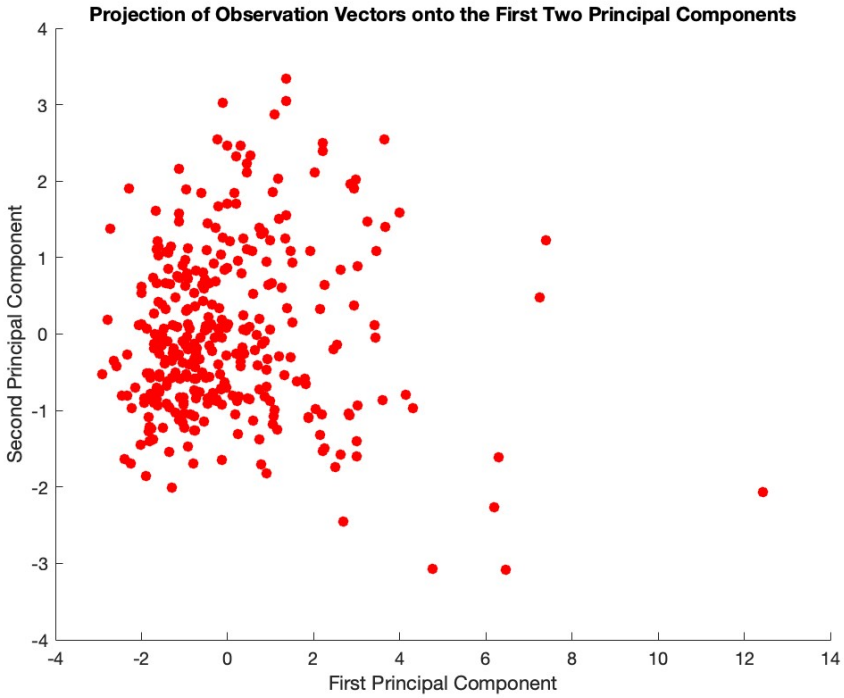
```
1  Captured variance by first 1 principal components: 0.3787
2  Captured variance by first 2 principal components: 0.5136
3  Captured variance by first 3 principal components: 0.6404
4  Captured variance by first 4 principal components: 0.7427
5  Captured variance by first 5 principal components: 0.8264
6  Captured variance by first 6 principal components: 0.8965
7  Captured variance by first 7 principal components: 0.9513
8  Captured variance by first 8 principal components: 0.9866
9  Captured variance by first 9 principal components: 1.0000
```

4. Project all the observation vectors onto the subspace spanned by the first two principal components and use the appropriate MATLAB command to plot the projected vectors.

For this part, we use this MATLAB code:

```
1  % 4. Project onto subspace spanned by first two principal
       components
2  % (columns of matrix U from the SVD)
3
4  projections_svd = U(:, 1:2)'*X_hat; % projection onto subspace
5  PC1 = projections_svd(1, :); PC2 = projections_svd(2, :);
6  figure(1);
7  scatter(PC1, PC2, 'filled', 'r');
8  xlabel('First Principal Component');
9  ylabel('Second Principal Component');
10 title('Projection of Observation Vectors onto the First Two
       Principal Components');
```

which produces the following plot of the projection onto the subspace spanned by the first two principal components:



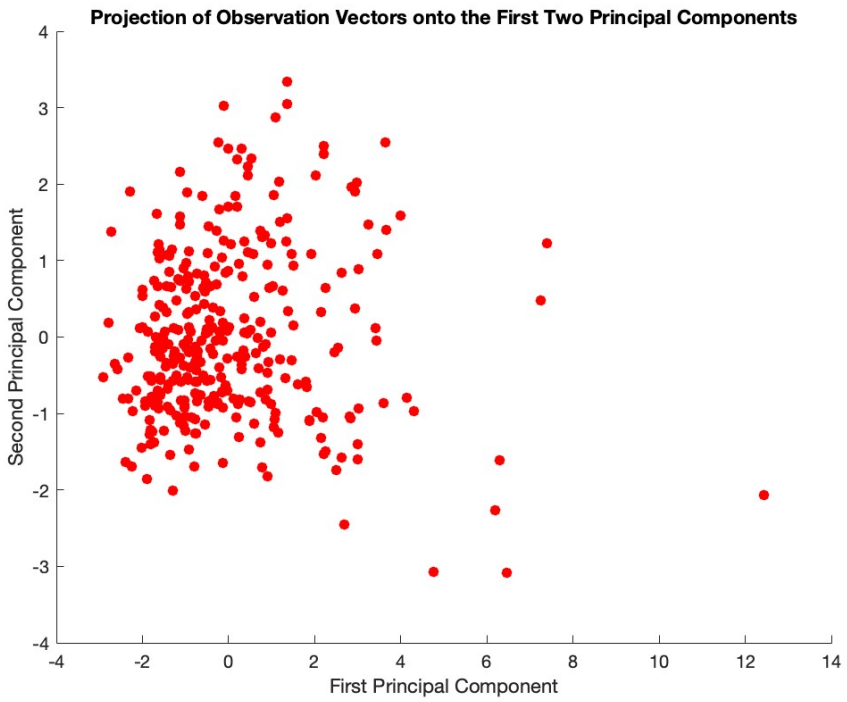
5. Use the MATLAB command `pca` to obtain the projected vectors of the precedent item and compare with the previous result.

```
% 5. Comparison with MATLAB pca command

[coeff, score, latent] = pca(X_hat');  % transposed because
   data observations need to be in rows, features in columns
projections_pca = score(:, 1:2)'; % Transpose for comparison
   with earlier results
projections_pca(2, :) = - projections_pca(2, :); % sign needed
   to match

figure(2);
scatter(projections_pca(1, :), projections_pca(2, :), 'filled',
    'r');
xlabel('First Principal Component');
ylabel('Second Principal Component');
title('Projection of Observation Vectors onto the First Two
   Principal Components');
```

We note that a sign flip in the second component is required for the results to match the previous ones. The reason for this is not clear, and we attribute this to the inner workings of MATLAB. This gives the following plot:



We see that this plot using the MATLAB command `pca` looks identical to the previous one, for which we computed the SVD directly, confirming the validity of our method. To verify this, we compute the norm of the difference of the matrices of projected components:

```
1 difference = norm(projections_svd - projections_pca);
```

giving a result of 1.28×10^{-13}.